StyleSync: AI-Driven Personalized Outfit and Shopping Assistant

Parth Keyur Gawande (pg5598@rit.edu)

Table of Contents

Table of Figures

Section 1 – Introduction

Fashion is an expressive, dynamic force of identity, culture, and personal creativity. As e-commerce websites expand and fashion trends become globalized, consumers are presented with a myriad of often bewildering fashion choices. Such an extensive range, while empowering, tends to result in decision fatigue, making it challenging for consumers to create outfits that are fashionable and coherent. As such, most wardrobes contain unused or unmatched items, resulting in wasted spending and dissatisfaction with personal style choices.

In response to these difficulties, the infusion of artificial intelligence (AI) and deep learning technologies has changed the way individuals engage with fashion. AI-powered solutions now facilitate style discovery, trend prediction, and virtual styling, providing users with personalized recommendations based on visual and contextual hints. Nevertheless, whereas most current systems are very good at recommending individual items, they fall short in the more difficult problem of determining outfit compatibility — the probability that a set of items will hang together well as an overall ensemble.

This paper presents StyleSync: An AI-Powered Personalized Outfit and Shopping Assistant, which fills this important void. Rather than suggesting separate articles of clothing based on superficial visual similarity, StyleSync considers whole outfits holistically. Users can upload photos of several articles of clothing — tops, bottoms, shoes, bags, and accessories — and the system predicts how well the items complement each other. When it finds inconsistencies or mismatches in style, StyleSync provides suggestions for improvement and even proposes external options through dynamic product retrieval from live e-commerce websites.

At the core of StyleSync is a Multi-Layered Comparison Network (MCN) architecture-based deep learning model that forecasts outfit compatibility by comparing relations among pairs of items at multiple levels of visual abstraction. Features are drawn from various layers of a ResNet-50 backbone network, ranging from fine-grained textures to high-level style semantics. Pairwise similarity is calculated among projected feature representations, and an overall compatibility score is produced to depict the cohesiveness of the outfit.

The system also has an interactive web-based UI built using the Dash framework, where users can upload outfits interactively, see compatibility, detect weak links, and get shopping suggestions. External API integration, such as via SerpAPI, also enhances the user experience by fetching live fashion product suggestions that are aligned with the user's style and category requirements.

By combining computer vision, semantic analysis, and real-time user interaction, StyleSync offers a practical and intelligent solution to modern wardrobe management. It aims not only to enable users to make better-informed styling choices, but also to assist in sustainable fashion by promoting the utilization of already owned garments.

The remainder of this report is organized as follows: Section 2 discusses related research in AI-powered fashion recommendation systems. Section 3 outlines the methodology, covering dataset preparation, model architecture, and system integration. Section 4 reports experiments and results obtained, and Section 5 concludes with major findings and possible avenues for future improvement.

Section 2 – Prior Work

The incorporation of artificial intelligence (AI) and deep learning within fashion recommendatory systems has played a vital role in revolutionizing how users engage with their wardrobe and identify matching apparel. Shirkhani et al. [1] provides one of the earliest detailed reviews of this field, synthesizing the critique of the development of AI-powered fashion recommenders. They categorize current models into three broad types: content-based filtering, collaborative filtering, and hybrid models. Content-based models focus on visual and text-based features of clothing, including texture, color, and style, whereas collaborative filtering uses historical user behavior to recommend products with identical features. Hybrid models try to combine these roles to improve recommendation quality. The authors highlight the central contribution of deep learning models, and in particular convolutional neural networks (CNNs), to feature extraction and compatibility modeling. Their results validate that models that combine computer vision with behavioral analytics resoundingly beat conventional rule-based systems by providing more pertinent, user-oriented outfit recommendations.

Zhang and Caverlee [2] address utilizing social media as a dynamic influencing factor for fashion recommendation. They highlight that fashion trends are likely to be influenced by cultural evolution, influencer promotion, and viral posts. The paper introduces an RNN-based model that embraces social media signals such as trending hashtags and influencer data to dynamically update fashion recommendations. One of the primary conclusions of their work is that static datasets are not adequate to capture fashion's fluidic nature. Thus, systems need to be live and context-sensitive and continuously update based on external social stimuli. In this manner, the approach has specific relevance to modern AI systems that are struggling to stay culturally and contextually relevant.

Suvarna and Balakrishna [3] propose an ensemble deep learning model-based robust content-based recommendation approach. Their model ensembles the predictions of MobileNet, DenseNet, Xception, and two versions of VGG through a weighted ensemble classifier. The model is trained and evaluated on real-world fashion datasets and attains 96% accuracy in classification. They utilize cosine similarity for item retrieval so that the recommendations are visually and stylistically coherent. The power of this ensemble method is in generalizing across a wide range of types of garments and for various textures and visual attributes. Feature diversity is also highlighted in their work as playing a significant role in recommendation accuracy, which is very much in agreement with the multi-layered strategy followed in StyleSync.

Lin et al. [4] presents the Neural Outfit Recommendation (NOR) system, a novel combination of outfit matching and explainable comment generation. Their method leverages CNNs to learn visual features from fashion item images and couples these with a mutual attention mechanism and a GRU-based language model in order to output user-friendly explanations. NOR, trained on the Polyvore dataset, not only yields high compatibility prediction accuracy but also generates coherent and contextually relevant natural language explanations for recommendations. Their focus on explainability aligns with the recommendation layer in GPT-based StyleSync, where the system not only recommends but also explains why the look is good or how it can be improved.

Lin et al. [5] present another important work with a category-based subspace attention network that is scalable for complementary fashion item retrieval. Unlike earlier works, which focused solely on predicting compatibility, their model extends to retrieving the best missing item to complement an outfit. The model makes use of attention mechanisms per clothing category (top, shoe, bag) and learns fine-grained attributes such as pattern, material, and occasion. Their training dataset, which is based on a large dataset of Polyvore, allows the system to identify complex inter-item relations and fill-in-the-blank (FITB) tasks more accurately. This work bears

strong resemblance to the item substitution and missing item suggestion components of StyleSync.

Kalinin et al. [6] presents a Generative AI-based fashion system that combines YOLOv8 object detection and GPT-4.0 Vision to produce customized fashion suggestions. They demonstrate how the vision and language models work together, employing an extremely large DeepFashion2 dataset to train the object detector and passing the detected clothing to GPT to generate suggestions. They even apply the system to actual fashion events and measure user satisfaction with the suggestions generated. Use of big language models for personalized fashion suggestions, as shown in this study, is at the forefront of how StyleSync employs GPT-3.5 to make fashion suggestions through text.

Wang et al. [7], the authors of the initial paper on Multi-Layered Comparison Network (MCN) upon which StyleSync is built, present a deep learning framework for predicting compatibility of outfits through comparing pairs of items across different semantic layers of a CNN. Unlike models that rely exclusively on the last CNN layer, MCN learns from the middle layers (conv2_x to conv5_x in ResNet-50), acquiring low-level characteristics like color and texture, as well as high-level features like style and shape. Projected embeddings and type-similar similarity masks are used in learning the relationships between different types of garments. Its AUC and FITB scores are better than baseline options like pooling, attention, and LSTM approaches. Additionally, it incorporates a gradient-based diagnosis process that identifies the weakest component in an ensemble to enable targeted replacement. These ideas are inherited directly and used in StyleSync through the addition of GPT-generated styling recommendations and SerpAPI-driven real-time product recommendations.

Cui et al. [8] introduce a new model called Correlation-aware Cross-modal Attention Network (C2ANet) to address the challenges of fashion compatibility modeling. Their approach enhances outfit compatibility predictions by both multimodal information (visual, text, and category information) and intra-outfit contextual correlations. Specifically, they propose a modality-driven collaborative learning module that involves multiple modalities in an end-to-end manner through cross-modal co-attention mechanisms, which guarantees semantic consistency and complementary relationships among the modalities. In addition, an information aggregation module that is aware of correlation is employed using graph attention networks to learn the intricate item-to-item relationships in ensembles adaptively. Extensive experiments on the Polyvore Outfits-ND and Polyvore Outfits-D datasets demonstrate that C2ANet outperforms a number of state-of-the-art methods on compatibility prediction and fill-in-the-blank tasks, wherein the importance of cross-modal fusion and contextual learning in fashion recommendation systems is also unveiled.

Han et al. [9] proposes a novel fashion recommendation system that learns outfit compatibility using a bidirectional LSTM (Bi-LSTM) model. Outfits, in their work, are considered sequential data where each item is a time step that reflects the natural dressing sequence (from tops to accessories). The model learns to predict the next item in the sequence from items it has seen so far, hence learning the compatibility relationships between different fashion categories. Apart from sequence learning, a visual-semantic embedding space is also jointly trained to embed item attributes and category information, enabling multimodal input capabilities. Their experiments on a newly collected Polyvore dataset demonstrate that this approach significantly outperforms previous approaches to fill-in-the-blank recommendation, full outfit generation, and compatibility prediction tasks. This paper highlights the importance of both sequential reasoning and multimodal representation learning for fashion recommendation systems.

Section 3 – Methodology

The approach used in this project was aimed at creating a robust image-based compatibility prediction system that can evaluate fashion ensembles and suggesting enhancements through web-based product suggestions. The starting point of the system is adapted from the system suggested in the Multi-Layered Comparison Network (MCN) of Wang et al [7] that compares item pairs within a set of items across different layers of visual abstraction. In this project, MCN architecture was reworked and extended to handle the entire Polyvore dataset and optimized for deployment with a real-time user interface and retail product recommendation via API integration.

This section splits the whole workflow into six subsections. Section 3.1 outlines the origin of the dataset, its composition, and preprocessing activities. Section 3.2 talks about the CNN-based feature extraction process via ResNet-50. Section 3.3 illustrates the Multi-Layered Visual Comparison Module. Section 3.4 presents compatibility learning, predicting and training fashion coherence, Section 3.5 outlines the interactive user interface implemented for the project, and Section 3.6 illustrates the way the system communicates with external APIs to fetch shopping recommendations for missing or mismatched products

3.1 Dataset Description and Preprocessing

The methodology for this project shifted significantly as it evolved from a proof-of-concept built on a small subset of data to a robust system trained on a large dataset. This section reports on the datasets used at both levels, recording structure, content, and rationale behind the selection of the final dataset.

3.1.1 Initial Dataset (Subset Phase)

During the initial stage of this project, I employed the Polyvore Outfit Dataset [10] hosted at Kaggle. The dataset is geared towards assisting fashion compatibility prediction and fill-in-the-blank (FITB) tasks and is suitable for building and evaluating outfit recommendation systems at a theoretical level.

A. Dataset Structure:

The dataset is organized into two primary folders:

- disjoint/: Contains training, validation, and testing data where the outfits used for training have no overlapping items with those used for testing. This strict separation ensures that evaluation measures true generalization.
- nondisjoint/: Allows item to overlap across training and testing outfits, which can facilitate quicker model convergence but introduces some bias.

  Each folder contains the following key files:

- compatibility_train.txt, compatibility_valid.txt, compatibility_test.txt:
  These files list outfits by providing the item image filenames that make up each outfit. Each line represents a complete outfit (e.g., 110264109.jpg 106086121.jpg 110605937.jpg).
- fill_in_blank_test.json and fill_in_blank_valid.json:
  These files define FITB tasks where an incomplete outfit is paired with four candidate options to predict the best matching missing item.

- polyvore_item_metadata.json:
  Provides metadata for each fashion item, including its ID, category label, image URL, and descriptive name.
- polyvore_outfit_titles.json:
  Contains metadata at the outfit level, such as the outfit's name, creator, and creation date.
- category_id.txt:
  A simple mapping from category IDs to category names.

B. Data Selection and Preprocessing:

Given hardware and memory constraints during the initial experimentation phase, only a limited sample size was extracted:

- Training Set: 500 outfits sampled from compatibility_train.txt
- Validation Set: 500 outfits sampled from compatibility_valid.txt
- Test Set: 500 outfits sampled from compatibility_test.txt

The corresponding images were retrieved from the images/ directory, resized to 224x224 pixels, and normalized according to ImageNet preprocessing standards. Outfits with missing images were filtered out during preprocessing to avoid loading errors during model training

C. Limitations Encountered

Although the original subset of the Polyvore Outfit Dataset was useful in developing a preliminary pipeline and testing fundamental functionalities such as feature extraction, batching of outfits, and compatibility scoring, it proved to have some essential limitations:

- Limited Diversity: Since there were only 500 samples per split, the dataset did not possess an adequate diversity of item categories, colors, materials, and styles, making the model's learning space limited.
- Poor Generalization: After being trained on this small dataset, the model was frequently not able to consistently make compatibility judgments on new or novel combinations of outfits.
- Limited Outfit Complexity: Many of the sampled outfits were straightforward (e.g., top-bottom only) and didn't involve more complex multi-item relationships that would be found in actual styling scenarios (e.g., coordinating tops, bottoms, shoes, and accessories).
- Sparse Metadata Use: While metadata like category information and outfit names were present, they were not closely coupled with the training data (compatibility_train.txt only had image filenames). This made it challenging to incorporate category-specific reasoning in initial versions of the model.

Due to these constraints, the project was forced to change over to a larger data set that would be capable of yielding a higher richness of ensembles, adding more item diversity, and better supporting real-world fashion compatibility modeling. The project therefore changed over to the Maryland Polyvore Images Dataset, which is discussed in detail in the subsequent section.

3.1.2 Final Dataset Phase: Maryland Polyvore Images Dataset

For the final phase of model development, the Maryland Polyvore Images Dataset [12] was adopted. This data set provides a large-scale, organized, and clean, structured collection of fashion outfits, making it an ideal choice for training and evaluating a robust compatibility prediction model. The dataset is built upon fashion outfits curated by users of the Polyvore

platform and has been refined specifically for academic research following standards established in earlier works [7], [11].

The data set comprises:

- 33,375 outfit directories, each corresponding to a unique outfit.
- 444,371 individual fashion item images, carefully cropped and formatted with plain backgrounds to focus on the visual properties of clothing items.

Each outfit is stored in a dedicated folder named using a numeric outfit ID, such as images/201969694/. Within each outfit folder, individual clothing items are saved as image files, indexed numerically (e.g., 1.jpg, 2.jpg, 3.jpg), with each file representing one specific item in the outfit.

A. Annotation Files

The dataset is annotated using three key JSON files:

- train_no_dup_with_category_3more_name.json
- valid_no_dup_with_category_3more_name.json
- test_no_dup_with_category_3more_name.json

Each JSON file maps an outfit ID to a dictionary of item categories, providing structured information about what clothing items belong to which outfit and their respective positions.

A typical JSON entry looks like the following, as given in Figure 1:


```
"201969694": {
  "upper": {"index": 1, "name": "new look light blue denim oversized long sleeve shirt"},
  "bottom": {"index": 2, "name": "mango skinny jane jegging"},
  "shoe": {"index": 3, "name": "sophia webster leather butterfly flats"},
  "bag": {"index": 4, "name": "michael michael kors mini selma crossbody bag"},
  "accessory": {"index": 5, "name": "velvet vase embellished necklace"}
}
```


Figure 1: Example of Outfit Annotation in Polyvore JSON File

From this example:

- Upper: Refers to tops, shirts, jackets, or outerwear items.
- Bottom: Includes jeans, skirts, trousers, and shorts.
- Shoe: Encompasses various types of footwear, including sneakers, boots, flats, and heels.
- Bag: Represents bags like totes, clutches, and backpacks.
- Accessory: Covers items such as jewelry, scarves, belts, and sunglasses.

Each item is associated with:

- An index (integer), pointing to the image filename (e.g., 1.jpg for the upper item).
- A name (string), providing a descriptive label for the fashion item.

This structured labeling is critical for organizing the items within an outfit and for associating each visual input with its category during model training.

B. Fashion Item Categories

Across all outfits, the items are categorized into five major groups:

- Upper: Tops, blouses, jackets, sweaters, outerwear.
- Bottom: Jeans, skirts, trousers, shorts.
- Shoe: Sneakers, boots, sandals, heels.
- Bag: Handbags, crossbody bags, clutches, totes.
- Accessory: Jewelry, sunglasses, hats, belts, scarves.

Not all outfits necessarily contain all five categories. Some may be incomplete, missing an accessory or a bag, which adds to the dataset's realism by representing how people style outfits in flexible, non-standardized ways.

C. Organization and Image Structure

The images themselves are systematically stored to facilitate efficient loading and access:

- Each outfit folder contains image files named as 1.jpg, 2.jpg, 3.jpg, etc., matching the indices provided in the JSON annotations.
- The images are cropped product photos with plain or minimalistic backgrounds to avoid distracting elements and focus on the clothing itself.
- All images are standardized in terms of aspect ratio and resolution before feature extraction (details covered in Section 3.2).

The presence of both complete and partial outfits makes this dataset particularly suitable for learning fashion compatibility patterns, diagnosing incomplete looks, and suggesting missing pieces — all of which are essential real-world styling scenarios

D. Dataset Split Information

The dataset is split into training, validation, and testing sets without overlapping outfits between the splits:

- Training Set:
    - Approximately 19,780 outfits
    - Includes a broad mix of everyday and special occasion styles.
- Validation Set:
    - Approximately 2,961 outfits
    - Used to tune model hyperparameters and prevent overfitting.
- Test Set:
    - Approximately 5,212 outfits
    - Reserved for final evaluation and performance measurement.

This structured partitioning ensures a fair and unbiased assessment of the model's ability to generalize beyond the examples it was trained on.

3.2 Preprocessing Workflow and Feature Extraction

Following the structuring of the dataset, the preprocessing of the fashion product images for modeling compatibility and feature extraction was the next task. It consisted of an efficient image preprocessing pipeline, effective data loading procedures, and computation of semantically relevant visual embeddings via a pretrained convolutional neural network (CNN).

3.2.1 Image Preprocessing

All item images, which were initially in raw JPG format, were run through a standard preprocessing pipeline to have consistency throughout the training, validation, and test processes. Preprocessing steps were designed to adapt to the ResNet-50 [13] backbone used later for feature extraction.

The primary preprocessing steps included:

- Resizing: All images of the items were resized to 224×224 pixels. The resizing was done to meet the input size requirement of the ResNet-50 model so that there is no distortion or aspect ratio difference introduced in different fashion item categories.

- Normalization: Each image was normalized using the channel-wise mean and standard deviation values from the ImageNet dataset:

    - Mean: (0.485, 0.456, 0.406)
    - Standard Deviation: (0.229, 0.224, 0.225)

  Normalization helped to tighten pixel value distributions for stability in model feature extraction along with allowing faster convergence during training.

- Missing Image Filtering: An entire ensemble was skipped during data loading if an ensemble contained missing or unreadable item images. The filtering allowed batch processing integrity to avoid feeding the model with incomplete or distorted input batches.

- Category Standardization: In the interest of consistency across varied outfits, certain categories were grouped together:

    - Outerwear items (such as jackets and blazers) were marked under Upper.
    - Accessories such as scarves, belts, jewelry, and sunglasses were all marked under Accessory.

  This grouping forced each outfit into a five-category model: Upper, Bottom, Shoe, Bag, and Accessory, thus ensuring consistency for subsequent feature extraction and pair-wise comparison.

3.2.2 Data Loading and Batch Structuring

We used an in-house PyTorch Dataset class, to handle the loading of train and test data. The responsibilities of the loader were:

- JSON annotation file parsing to load outfit-item relationships.
- Constructing item image file paths dynamically by outfit ID and image index.
- Enforcing resizing and normalizing transformation on each image loaded.
- Batching all the ensembles together in one batch, with the same internal ordering as Upper → Bottom → Shoe → Bag → Accessory.

Maintaining this strict internal ordering was critical because the model's pairwise comparison module assumes that specific item types are compared systematically, preserving the logical relationships among outfit components.

3.2.3 Feature Extraction Using ResNet-50

After pre-processing, images were further passed on through pre-trained ResNet-50 network [7] in order to achieve high-dimensional visual embeddings. As opposed to making use of only the final classification layer, middle-level convolution layers were employed for richer understanding of visual feature sets of an object.

The feature extraction process included:

- Multi-Level Feature Extraction: Features were sampled from different convolutional blocks of ResNet-50.Lower layers captured low-level features like edges, textures, and patterns, while deeper layers retained higher-level features like the overall style, shape, and category-specific signals.
- Global Average Pooling (GAP): Each selected convolutional output was used with a Global Average Pooling. This averaged the spatial feature maps to dense vectors without semantic loss.
- Embedding Generation: Multi-layered aggregated outputs provided diverse feature vectors for every item at different levels of abstraction. These were then used in the comparison module to check compatibility of an outfit (described in Section 3.3).

Multiple-layer feature extraction ensured that high-level visual similarities (e.g., texture of cloth) and higher-level stylistic similarities (e.g., matching casualness or color tone) were obtained in an efficient manner.

3.3 Multi-Layered Visual Comparison Module

Fashion compatibility assessment requires considering more than one thing at a time; it requires understanding how several visual things work together on a whole outfit. The Multi-Layered Visual Comparison Module caters to this richness by rigorously extracting features at multiple levels of abstraction, mapping them for meaningful cross-category comparison, computing fine-grained pairwise similarities, and finally predicting a unified outfit compatibility score. This part consists of a step-by-step tour of each step of the module, explaining its theoretical basis and concrete impact on the fashion compatibility task.

3.3.1 Feature Extraction at Multiple CNN Layers

Hierarchical representations are acquired by deep convolutional networks like ResNet-50 across layers. Basic low-level visual features like edges, colors, and textures are typically handled by early layers. Mid-level layers are responsible for material structure, patterns, and repeated motifs, while lower layers contain abstract semantic information that determines larger categories like "sneaker" or "blazer" and even style categories like "casual" or "formal."

In this project, feature extraction from a number of mid-level layers allows the model to reason about multiple perceptual levels of compatibility simultaneously. Lower levels are primarily responsible for controlling material and color harmony, such that these matter when harmonizing textures like satin or leather on garments. Mid-levels facilitate the harmonizing of structural silhouettes of garment pieces, whereas upper layers harmonize the overall theme or formality level of an outfit. This multi-resolution feature capturing is quite similar to how human beings think of fashion, fine details and general atmosphere being given consideration simultaneously.3.3.2 Global Average Pooling (GAP) for Feature Condensation

Following the extraction of feature maps from each selected CNN layer, they are then condensed into concise vectors with the help of Global Average Pooling (GAP). GAP operates by averaging activation values along spatial axes of each feature map channel, thereby abstracting "what"

feature is present without emphasizing "where" it is. Spatial invariance is particularly well-suited to fashion images, where the style or material of an item is important but not its exact location within the frame.

In this work, GAP ensures that the model is focused on content features like color, texture, and style rather than positional biases, which could otherwise bias the compatibility estimation. GAP also reduces the feature representation size, saving model parameters and computation overhead while preserving important semantic information necessary for deep relational reasoning.

3.3.3 Type-Aware Feature Projection

The products in various categories such as tops, shoes, and bags naturally differ in visual appearance and feature distributions. For feasible comparison across categories, the model applies type-specific projection transformations over the features learned from each product. These projection layers learnable by the model transform the item-type-based embeddings so that their representations are aligned into a shared latent space where pair-wise comparison is meaningful. By offering type-specified projections, the model precludes the hazard of compatibility judgment based on innate disparity between object types.

For example, it prohibits a fashionable boot from being unfavorably compared to a soft jacket, focusing instead on compatible style cues like roughness or streamlining over misleading geometric features. Normalization is essential to stylistically coherent compatibility prediction, unlike simply relying on superficial surface look similarity.

3.3.4 Pairwise Similarity Computation and Multi-Level Reasoning

After projecting the item features into a shared latent space, the model proceeds to systematically enumerate all the pairs of items in an outfit and compute pairwise similarities for each. Cosine similarity is used as a metric to measure to what degree two items are correlated in the embedding space. Specifically, the similarity calculations are conducted independently at different abstraction levels : low, mid, and high, for different CNN feature layers. Multi-level comparison makes the model more familiar with outfit compatibility.

At low-level features, the model isolates basic visual attributes such as color coordination and texture match of materials like ensuring that a beige-colored shoe is color-compatible with a cream-colored handbag. At mid-level features, the model makes inferences of structural forms and material similarities, such as a fitted top paired with skinny jeans to attain visual balance. At high-level features, the model cares about general stylistic themes such as ensuring that a streetwear hoodie is properly outfitted with casual sneakers, maintaining thematic coherence.

By the combination of reasoning at these different levels of abstraction, the model constructs a stratified understanding of how individual pieces work with each other in an outfit. This mirrors the cognitive process employed by human stylists, who consider both tactile harmony (textured feel) and thematic coherence (formal vs. casual tone) in tandem when choosing an outfit that looks harmonious.

3.3.5 Aggregating Outfit Compatibility and Diagnostic Interpretation

Once the model determines all the pairwise similarity scores between the various abstraction layers, it adds these together to create a single, overall compatibility score for the outfit. A simple averaging method is typically used, where all item pairs are treated as equals. This addition acknowledges the fact that the visual success of an outfit is not the result of a single, prevailing item or pair, but rather of the total cohesion of all items. A set in which every item pair has high compatibility will, naturally, score high overall.

Conversely, even when items match up individually, having mismatched components will bring the final score down, assuming as it does real-world fashion judgment, where total visual balance matters most. The system not only provides prediction but also gradient-based diagnosis to diagnose the compatibility score.

At test time, the model computes each items and each pair of items' contribution by testing the sensitivity of the overall compatibility score to infinitesimal input perturbations. By backpropagating the gradients of the score to the pairwise similarities, the model identifies which relationships are net negative.

If removing a particular piece would significantly improve the overall compatibility score, then that piece is tagged as the least compatible part of the outfit. This enables not just passive scoring but also actionable feedback, showing which part of the outfit can be replaced to enhance coherence. The model thus acts as both a predictor of compatibility and an intelligent stylist assistant, giving users actionable feedback to enhance their outfit choice.

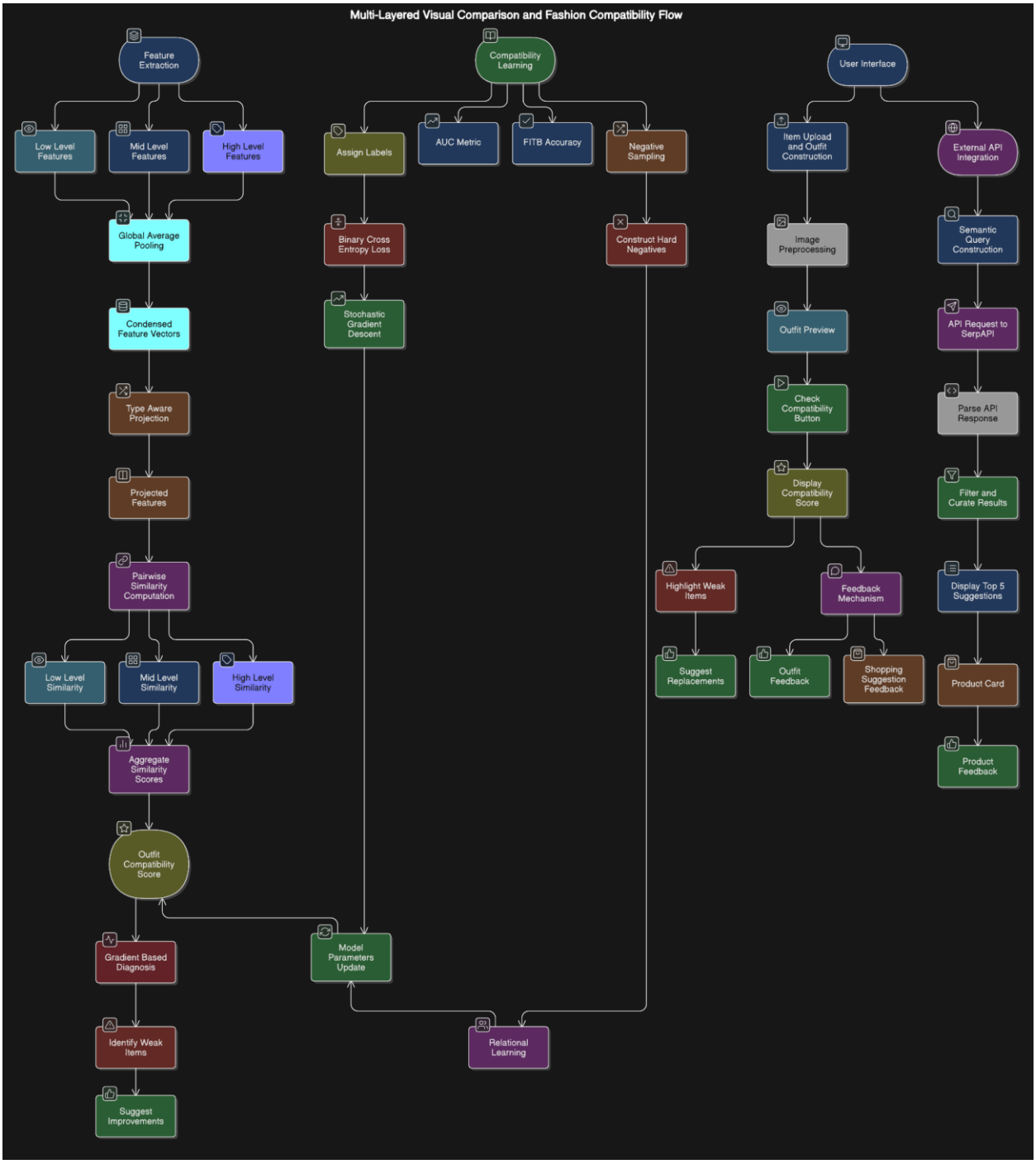

Figure 2: Detailed process flow showing multi-layered feature extraction, pairwise similarity computation, compatibility scoring, and diagnosis in StyleSync

To better visualize the step-by-step workflow of feature extraction, pairwise comparison, and overall compatibility scoring described in the Multi-Layered Comparison Module, the detailed process is illustrated in Figure 2. It captures the flow from low-level feature extraction to gradient-based diagnosis, showcasing the hierarchical reasoning structure used to evaluate outfit compatibility

3.4 Compatibility Learning: Predicting and Training Fashion Coherence

The fashion outfit compatibility prediction task is fundamentally framed as a supervised learning problem. In this setup, each outfit is associated with an explicit label; curated human-created outfits are labeled as 1 (compatible), while synthetically altered outfits are labeled as 0

(incompatible).

The model’s objective during training is to learn to distinguish compatible from incompatible outfits by analyzing relational visual cues extracted through the multi-layered comparison module. By leveraging direct supervision, the system is guided to align its judgments with human fashion sensibilities, rather than relying on unsupervised clustering or latent self-discovery.

Once the visual features of items are encoded through deep convolutional layers and their pairwise relationships computed, the learning phase focuses on associating these relational signals with appropriate compatibility scores. The sections that follow detail the loss functions, evaluation metrics, optimization techniques, and interpretability mechanisms that govern this learning process.

3.4.1 Defining Outfit Compatibility

The core value the model estimates is a compatibility score, measured as a continuous value between 0 and 1. It is higher for stronger stylistic and visual consistency among the items of the outfit, and lower when there are mismatches in color schemes, material textures, structural shapes, or overall style. This score is not derived from individual item properties; rather, it emerges from a combination of relational pairwise similarities computed at different semantic levels, low-level texture relations, mid-level material consistency, and high-level thematic consistency.

By maintaining this multi-layer relational structure, the compatibility score allows the system to mimic human-level fashion judgment, trading off explicit material examination with high-level stylistic reasoning.

3.4.2 Metrics and Loss Functions

Training the model to be able to differentiate between compatible and incompatible outfits highly relies on appropriately chosen metrics and loss functions.

The primary loss function used is the Binary Cross-Entropy (BCE) Loss, which is a typical approach for supervised binary classification problems. Binary Cross-Entropy calculates the divergence between the model's predicted compatibility score and the true label for each outfit. If the model finds the outfit to be genuinely compatible, then it is nudged to rate it near 1; if incompatible, near 0. Reducing BCE during training ensures not only that the model is making the right binary decision but does so confidently, a favorable property for such fine-grained aesthetic tasks as judging outfit compatibility.

To tune the model parameters, Stochastic Gradient Descent with momentum is used. SGD gives efficient mini-batch-based updates and is well suited for big datasets. Momentum also accelerates convergence by averaging noisy gradient updates such that the optimizer can move efficiently to lower regions of the loss surface.

At test time, two performance measures are used: Area Under the Receiver Operating Characteristic Curve (AUC) and Fill-in-the-Blank (FITB) Accuracy. AUC is a threshold-independent measure of the model's ability to discriminate between compatible and incompatible outfits across the entire range of score thresholds. It provides a good indication of the model's overall discrimination capability.

FITB accuracy quantifies how well the model can propose the best possible missing item to complete an incomplete look. This metric estimates real-world usage scenarios, where users are able to request suggestions for missing pieces in a partially constructed look.

BCE Loss, AUC, and FITB together offer a balanced configuration for training and testing a model that must reason relationally across several visual abstraction levels.

3.4.3 Negative Sampling for Improved Relational Learning

While positive examples are straightforward, capturing real-world fashion outfit compatibility, constructing helpful negative examples is a significant challenge. Rather than labeling a randomly chosen outfit as incompatible, negative examples are constructed by substituting one piece in an otherwise compatible outfit with an arbitrarily chosen item of the same type.

This invisible disturbance preserves the overall structure of the ensemble but introduces relational dissonance in terms of color, material, or theme. From these "hard negatives," the model catches the nuance of interaction among pieces of garment that define genuine stylistic harmony. By doing so, this approach avoids the pitfall in which models who learn solely from radically incompatible exemplars fail to develop more subtle relational aesthetics.

3.4.4 Compatibility Prediction and Interpretability

Upon model training, it calculates a single score of compatibility for any ensemble, measuring the degree to which the constituent pieces engage with one another on texture, material, shape, and style aspects. Besides generating scores, the model offers interpretability in the form of a gradient-based diagnosis method. By backpropagating the gradients of the pairwise similarities, the model is able to decide which specific items or pairs of items are contributing most to lowering the overall harmony of the ensemble.

Elements of an outfit contributing to maximum gradient to incompatibility are pointed out, giving actionable feedback on which an outfit can be adjusted, e.g., by changing an incompatible handbag or inappropriate shoes. This aspect of interpretability makes the model transition from being a black-box predictor to an active stylistic guide, which can rate and improve user-entered outfits with minimal adjustments.

3.5 Interactive User Interface and Feedback System

To provide smooth, user-friendly experience in working on the fashion compatibility model, an interactive web-based User Interface (UI) was developed in Python using the Dash framework. Dash Bootstrap Components were used to ensure that there was order in the layout and looked modern and professionally styled throughout the platform. The prime focus was to make it easy for individuals to create outfits, ensure compatibility, and leave comments, all of which with an interactive, good-looking interface.

The UI is organized into two major panels:

- Item Upload and Outfit Construction Panel: The users are able to upload images of fashion items of different categories of fashion items, such as Tops, Bottoms, Shoes, Bags, and Accessories. After uploading an item, it is displayed instantly in a specially set preview section, from where the user is able to look at the constructed outfit in advance before proceeding to compatibility checking. The uploaded images are regularly resized and preprocessed so that input remains uniform during model inference.

- Output and Recommendation Panel: As soon as the user chooses the "Check Compatibility" option, the model calculates the compatibility score for the chosen outfit and shows the output dynamically. In situations where some items are detrimental to the coherence of the outfit, the interface shows probable improvements and recommends changes based on internal diagnosis rationale. There is a complete decoupling between initial construction of the outfit and suggested changes to ensure transparency to the user.

The overall theme of the design is a dark appearance with blue highlight colors (#00BFFF) for better readability, along with rounded card shapes, slight hover effects, and responsive resizing on a range of devices.

Aside from checking for compatibility, the site also includes a Feedback Mechanism to allow for future system refinement. After receiving compatibility scores and improvement suggestions, users are prompted to provide quick feedback through Thumbs Up (👍) or Thumbs Down (👎) buttons.
This feedback is available at two levels:

- Outfit Compatibility Feedback: Users rate the model's suggestion for the entire outfit.
- Individual Shopping Suggestion Feedback: Users independently rate each external web-based product recommendation fetched through API integration.

While the feedback system has been fully implemented and incorporated into the user interaction loop, official user studies and feedback analysis were not conducted at this project stage. The feature is utilized as an infrastructure for future development to allow the system to gather data on user preferences to iteratively improve and potentially have personalization models.

In general, the feedback mechanisms and interactive UI are complementary and function together, enabling not only real-time fashion opinion but also supporting a foundation for ongoing user-initiated refinement to the prediction capabilities of the system.

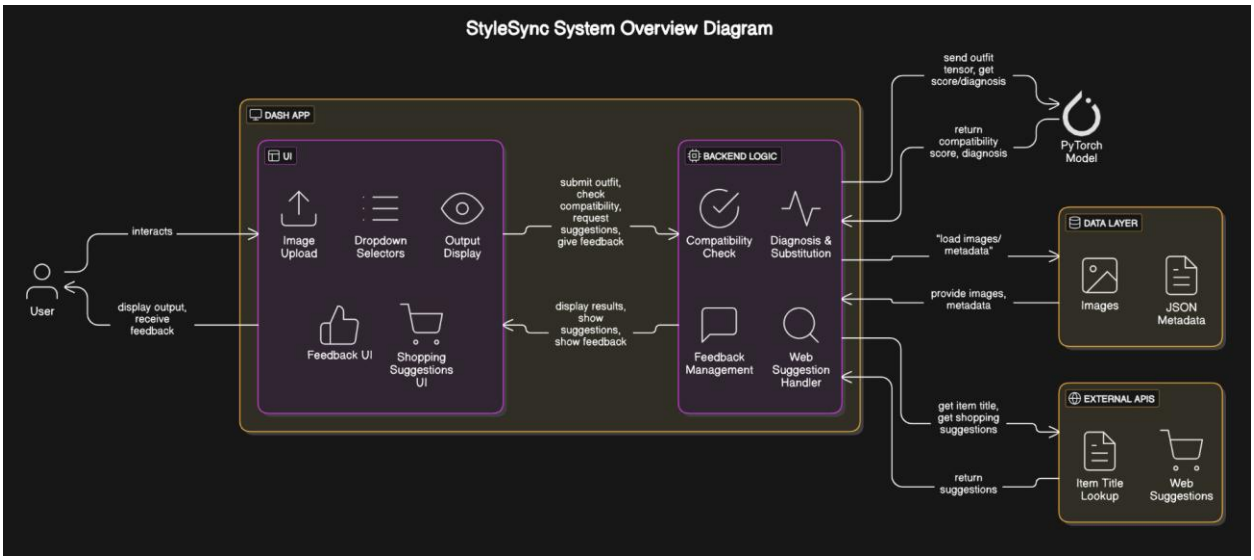

Figure 3: Overall architecture of the StyleSync system showing the UI elements, backend processing logic, PyTorch model interaction, and external API integration for dynamic recommendations.

The high-level system design for StyleSync, from user input to backend processing and external API communication, is captured in Figure 3. This architectural overview demonstrates the interaction between the Dash-based UI, the PyTorch model, the feedback loop, and dynamic web-based recommendations

3.6 External API Integration for Semantic-Based Retrieval

The system incorporates an External API Module centered on dynamic shopping recommendations to improve the user experience beyond internal compatibility scoring. The API integration bridges the gap between style assessment and practical buying advice by allowing customers to find new or replacement fashion products from live e-commerce sites.

SerpAPI, a real-time Google Shopping API that enables safe and effective querying for commercial fashion products, powers the external integration. Users are offered the opportunity to "Get Web Suggestions" through a dedicated button when the model's internal analysis reveals weak or missing outfit components.

The workflow for external retrieval involves several steps:

- Semantic Query Construction:
  Queries are intelligently formulated based on:
    - The clothing category (e.g., Tops, Shoes, Bags).
    - Descriptive attributes from the dataset (e.g., "white leather sneakers," "black crossbody handbag").
    - Optionally, user-provided textual prompts describing occasion or style preferences (e.g., "formal dress shoes" or "casual denim jackets").

  Semantic-based formulation ensures that the search reflects not just item type but also stylistic nuances, improving the relevance of the retrieved results.

- API Request and Result Handling:
  The formulated query is sent to SerpAPI through structured HTTP requests.
  Upon retrieval, the response is parsed to extract:
    - Product Title
    - Product Image (Thumbnail URL)
    - Product Link (Direct to E-commerce platform)

  Post-processing filters are applied to discard irrelevant entries and prioritize results matching the expected clothing type and styling intent.

- Display of Curated Recommendations:
  Only the Top 5 suggestions are displayed on the UI, using a clean card-based layout.
  Each card contains:
    - A thumbnail image
    - A direct, clickable shopping link
    - Independent thumbs-up/down buttons for feedback

The user is not overloaded with distracting or noisy product lists thanks to this architecture, which guarantees a targeted, customized purchasing experience. The following are some significant benefits of the semantic retrieval approach used here:

- Contextual Precision: The style and category requirements determined by outfit analysis closely match the retrieval queries.
- Dynamic Adaptability: User-generated prompts and occasion-based searching can be smoothly integrated into future improvements.
- User-Centric Curation: To ensure platform credibility, products are selected for quality and relevancy prior to presentation.

With the help of this external integration, the system transcends static outfit evaluation and offers users practical fashion guidance in addition to feedback, enabling them to achieve stylish enhancements.

Section 4 – Experiments and Results

This section provides a thorough description of experiments performed, results obtained, and analysis of model performance across various evaluation tasks. Every experiment is closely following the methodology described in Section 3, and each experiment only deals with multi-layered visual comparison, compatibility prediction, and external recommendation support.

4.1 Training Setup

The training phase of the StyleSync model began by initializing the network with pretrained weights derived from the architecture given by Wang et. al.[7]. The pretrained checkpoint file was based on training over a curated Polyvore dataset for fashion outfit compatibility tasks. Fine-tuning was then performed on the newly organized Maryland Polyvore Images Dataset [12], ensuring domain-specific adaptation to the updated outfit structures, item categories, and real-world styling scenarios.

The model was trained under the following settings:

- **Backbone Network:** ResNet-50 pretrained on ImageNet [13], adapted for compatibility evaluation.
- **Pretrained Weights:** MCN-based model checkpoint used for initialization to expedite convergence and enhance relational learning.
- **Input Size:** 224 × 224 pixels for all images.
- **Outfit Composition:** 3 to 5 items per outfit. Missing categories were padded with mean category images to preserve the input format.
- **Batch Size:** 28 outfits per mini batch.
- **Number of Epochs:** 55 epochs.
- **Optimizer:** Stochastic Gradient Descent (SGD) with momentum 0.9, ensuring smoother updates and faster convergence.
- **Initial Learning Rate:** 0.01, decayed by a factor of 0.2 every 10 epochs.
- **Loss Function:** Binary Cross-Entropy (BCE) Loss, designed for supervised binary classification between compatible and incompatible outfits.
- **Hardware:** NVIDIA GeForce RTX 4060 GPU with 8GB VRAM used for model training and fine-tuning.
- **Training Duration:** Approximately 9 to 10 hours for full convergence.
- **Model Saving:** Only the model checkpoint with the best validation performance was retained during training.

The training batches were constructed carefully to maintain a balance between positive examples (human-designed compatible outfits) and hard negative examples (outfits with intentionally mismatched items). This hard-negative sampling strategy improved the model's ability to distinguish subtle incompatibilities rather than only detecting obvious mismatches.

4.2 Evaluation Metrics

Two primary metrics were utilized to comprehensively assess model performance:

- Area Under the Receiver Operating Characteristic Curve (AUC): AUC measures the ability of the model to adequately classify compatible and incompatible combinations. Greater AUC indicates greater discriminative capacity across various thresholds.
- Fill-in-the-Blank (FITB) Accuracy: FITB simulates a real-world application situation where a user wants to complete a missing component of an outfit. With four options of a missing component, the model must select the most appropriate one for the rest of the

outfit. FITB evaluates both the model's compatibility reasoning and its ability to predict missing components.

Both measurements were tested five times on dynamically generated test sets, and mean results were provided to give immunity to sampling variance.

4.3 Compatibility Prediction Results

Several architectural variations were tested to verify the performance of the model for predicting outfit compatibility. Each model differed in how pairwise similarity scores computed by the Comparison Module (CM) were combined.

The major terms used here are:

- CSN (Conditional Similarity Network): A baseline metric learning method that projects items into a conditioned space for similarity comparison.
- CM (Comparison Module): Our core module computes direct pairwise similarities between projected item embeddings at multiple CNN layers.
- FC (Fully Connected) Layer: A dense neural network layer where each input is connected to every output. Here, multiple FC layers were stacked to allow the model to learn complex non-linear relationships from pairwise similarities.

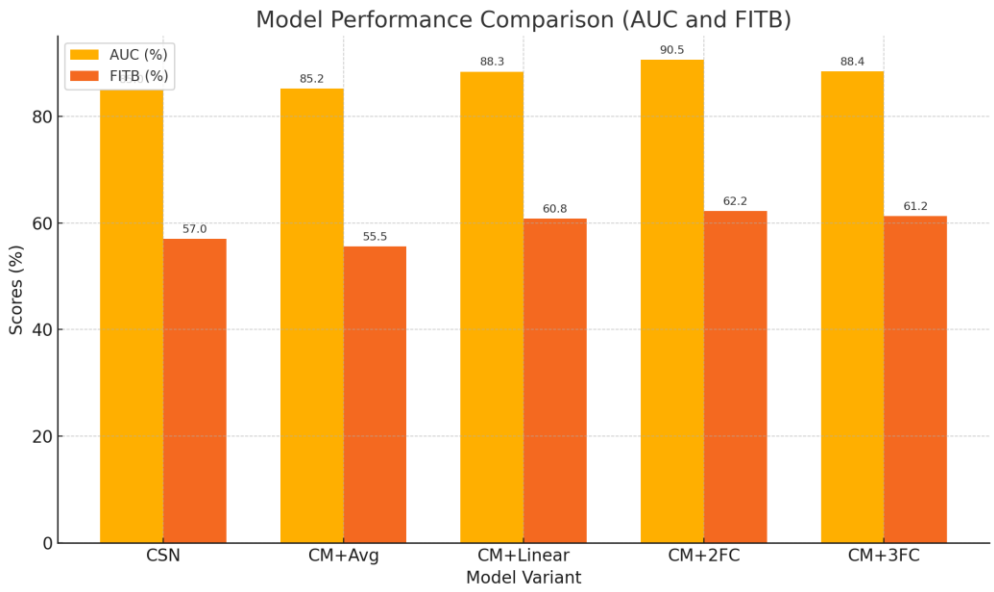

Figure 4: Comparison of AUC and FITB scores across different model configurations

Observations:

- CM + 2FC performed best with the highest AUC and FITB, confirming that aggregating pairwise similarities at more depths significantly improves compatibility reasoning, as seen in Figure 4.
- The Linear and overly non-linear 3FC versions did not perform as well as the 2-layer FC balance.

This validates the design choice to use two fully connected layers as a powerful but not computational-intensive predictor compared to learned similarity matrices

4.4 Multi-Layered Feature Contributions

To understand how different semantic levels affect compatibility learning, an ablation study was conducted. Different sets of CNN layers were used for feature extraction, and the corresponding results were tested accordingly. ResNet-50 layers extracted are described as below:

- Layer 1: The simplest textures and colors
- Layer 2: The simplest shapes and patterns
- Layer 3: Clothing shapes and material structures
- Layer 4: Visual semantics at higher levels like theme and style

The experimental setups tested combinations of these layers to analyze their impact, as shown in Figure 5.

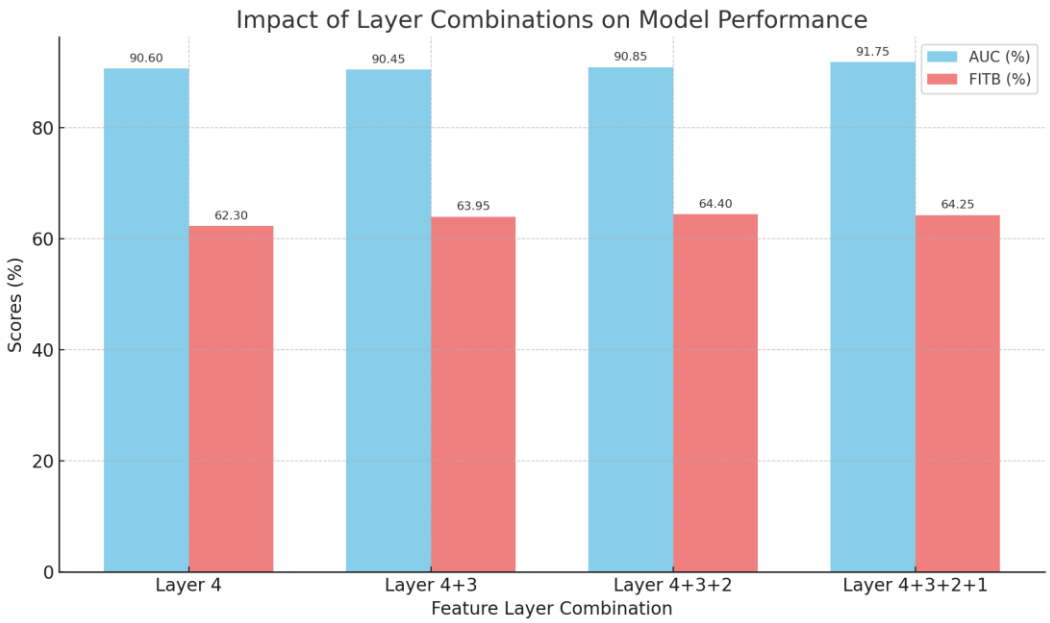

Figure 5: Effect of different CNN layer combinations on AUC and FITB performance

Key Learnings:

- Utilizing only Layer 4 provides a good style-level knowledge, but missing on texture-level facts.
- Including Layer 3 and 2 boosted compatibility scores by allowing the model to infer material consistency and structure coherence.
- Including Layer 1 captured surface texture and color coherence with minor improvement in performance at last.
- The best performance came when utilizing all four layers combined (4+3+2+1) for complete multi-level knowledge from textures up to styling.

This multi-level approach reflects human clothing evaluation: designers take into account fine texture (e.g., satin footwear), structure (e.g., narrow trousers with loose tops), and general theme (e.g., informal vs formal) simultaneously.

4.5 Outfit Diagnosis and Interpretability

Fashion compatibility is not simply a judgment of the look of individual garments but an appreciation of how the individual pieces work in relation to one another at multiple levels of visual abstraction. In an effort to provide more interpretable information in the compatibility

scores, the system uses a gradient-based diagnosis process that traces back what specific item pairs are responsible for high or low outfit coherence.

4.5.1 Layer-Wise Gradient Visualization

The diagnosis module computes backpropagated gradients over the pairwise similarity matrices at four different CNN layers that capture progressively deeper visual semantics:

- Layer 1: Grabs basic attributes like color, texture, and fine-grained material details.
- Layer 2: Grabs structural and material shape, i.e., garment silhouette and textile heaviness.
- Layer 3: Is interested in category-level semantics like discriminating "sneaker" vs "heels".
- Layer 4: Includes overall thematic coherence, such as in ensuring pieces are of a "formal" or "casual" category.

These multi-scale gradient visualizations not only allow for outfit compatibility to be predicted but also diagnose why an outfit fails or succeeds. In the following sections, some examples of outfits from the diagnosis results are shown, each exhibiting prominent failure or success modes, categorized according to properties at various abstraction levels.

4.5.2 Diagnosis of Incompatible Outfits

A. Example 1: Severe Incompatibility Due to Accessories and Shoes

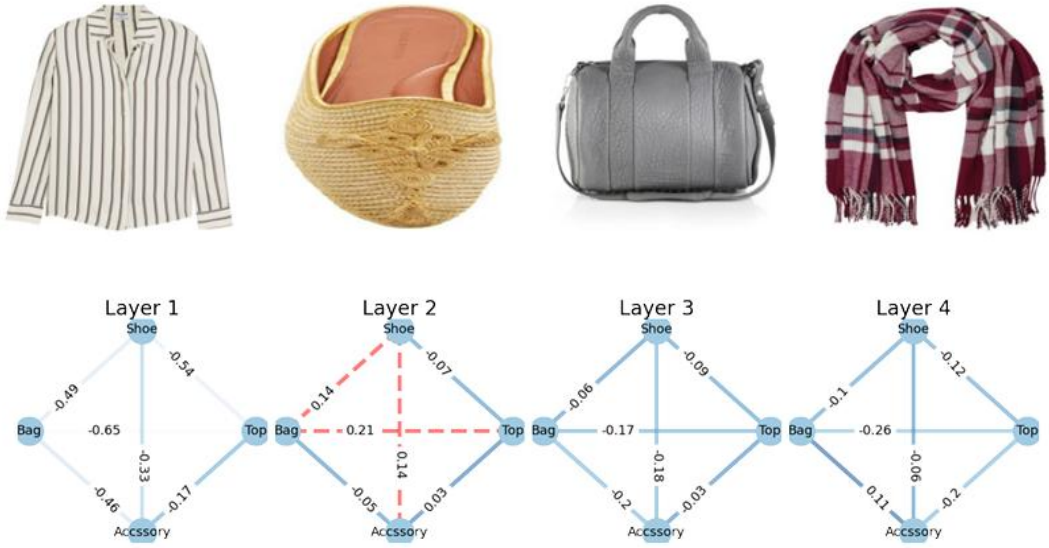

Figure 6: Diagnosis and Improvement for Outfit with Accessory and Shoe Conflict and Resolution

Predicted Compatibility Score: 0.0001

The gradients in Figure 6 suggested that the shoe material was in conflict with the top fabric at Layer 1, while the accessory caused a thematic discord at Layer 4. The accessory felt formal in nature, while the rest of the outfit remained casual, causing style dissonance.

Figure 7: Improved Outfit after substitution of shoes and accessories

After substitution, the score is 0.9692

After diagnosis of the failure, the shoe was replaced by a relaxed sneaker, and the accessory was replaced by a plain piece in line with the relaxed mood. As a result, the compatibility score of the outfit became much better to 0.9692 as can be seen from Figure 7, which attests that goal-driven substitution of peripheral pieces is able to fully restore outfit coherence.

B. Example 2: Structural Incompatibility Between Top and Bottom

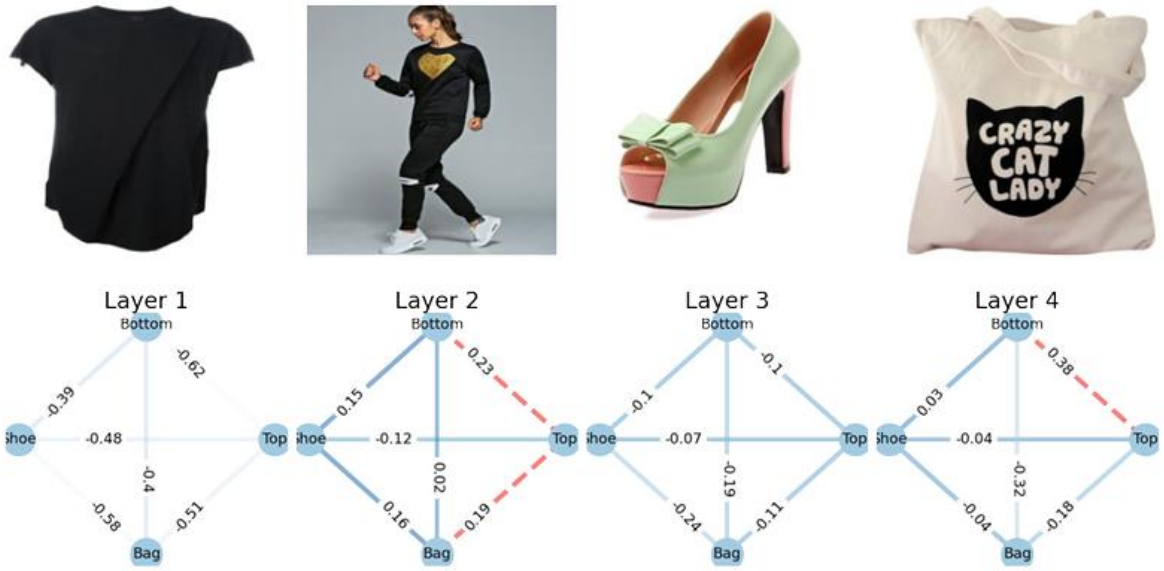

Figure 8: Diagnosis and Structural Adjustment for Outfit — Top and Bottom Misalignment

Predicted Compatibility Score: 0.0169

The big incompatibility resulted from Figure 8's material and silhouette discrepancy, bottom to top, noted primarily at Layer 2 and Layer 3. The bottom garment had a heavy, rough look incompatible with the light and smooth top garment, creating visual imbalance. Upon diagnosis, substituting the bottom with a less heavy, loose-fitting material (e.g., soft denim) significantly corrected outfit flow, and restructuring structural balance restored harmony through mid-level qualities.

C. Example 3: Major Incompatibility Driven by Bag and Shoe Conflict

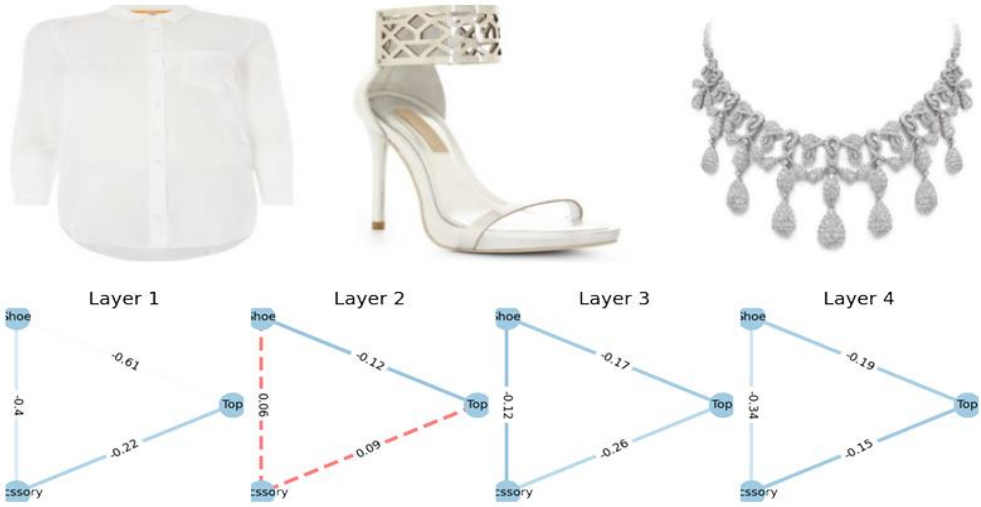

Figure 9: Diagnosis Visualization for Outfit with Bag and Shoe Texture and Theme Conflict

Predicted Compatibility Score: 0.0182

This outfit exhibited utter incompatibility at both low and high abstraction levels. Layer 1 gradients in Figure 9, ringed strong textural conflicts between the bottom object and shoes, showing that the material composition and surface texture of the two were highly discordant. Working up to Layer 4, the diagnosis found a thematic inconsistency brought by the bag, which radiated a more formal atmosphere compared to the otherwise casual outfit. With these low-level and high-level conflicts, the compatibility score was severely repressed to 0.0182. Replacing the bag with a casual crossbody and the shoes with fabric texture more akin to the pants could significantly improve the overall coherence of the outfit.

D. Example 4: Partial Compatibility with Minor Structural Misalignment

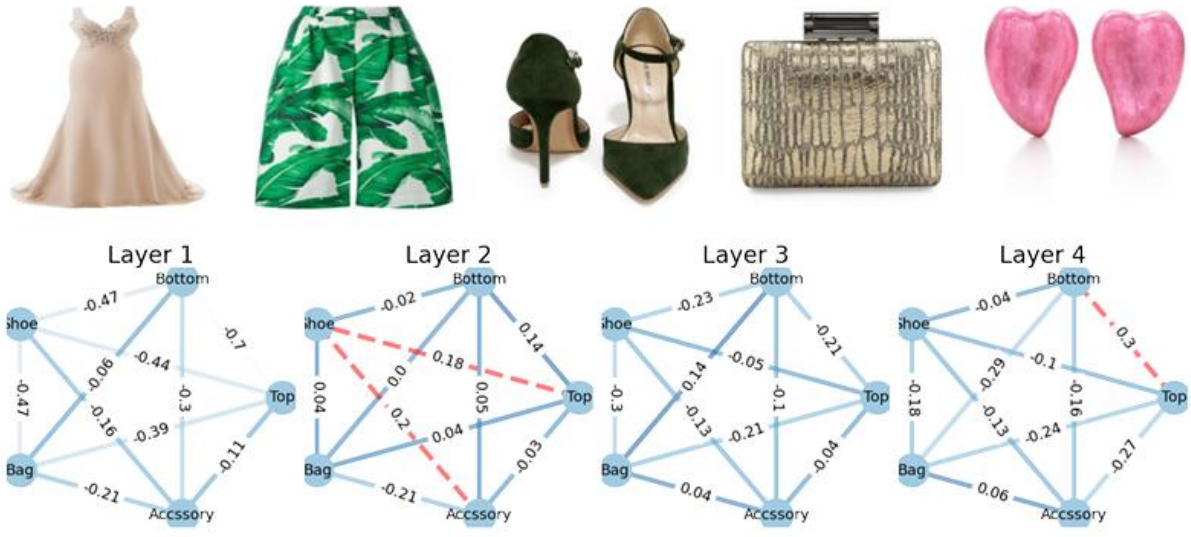

Figure 10: Diagnosis Visualization for Outfit with Mid-Level Structure and Material Mismatch

Predicted Compatibility Score: 0.4211

This outfit in Figure 10 exhibited partial compatibility, as indicated by a moderate value of 0.4211. The greatest inconsistency identified was at the Layer 2 middle-level feature maps, where minor misalignments in structure and weight of the fabric at the top, bottom, and accessories created an unbalanced visual flow. Although the overall theme was still quite intact, the diagnosis stated that structural harmony could be achieved by adjusting either the bottom piece or the accessories to better harmonize with the flowing nature of the top. This example serves to emphasize the importance of not only color compatibility but also material and form compatibility in obtaining overall outfit compatibility.

E. Example 3: Strong Compatibility Across Layers

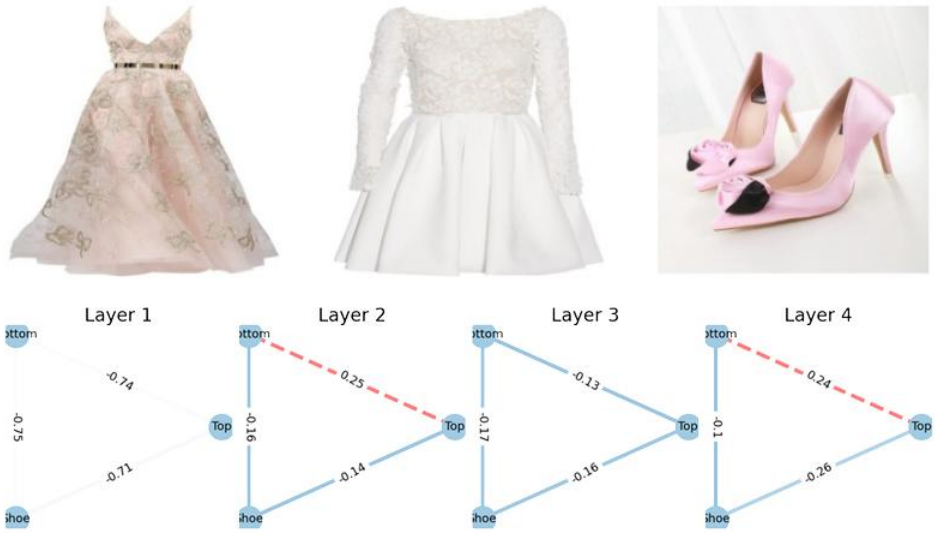

Figure 11: Diagnosis Visualization for Outfit — High Visual Harmony Across Texture, Structure, and Theme

Predicted Compatibility Score: 0.8115

This set of items reached a high level of compatibility of 0.8115, with all CNN feature layers having smooth visual coherence. Figure 11 indicates low-level color and texture harmonized smoothly, with minimal conflict at Layer 1, and Layer 2 and 3 claimed consistent material shapes and silhouettes for all the items. Most importantly, Layer 4 exhibited full stylistic coherence, with each item working together towards a cohesive, elegant appearance. No single piece was labeled as a disruptor element, indicating that harmony at micro (texture, fabric) and macro (style, theme) levels was appropriately preserved, making this collection a good positive reference point.

4.6 User Interface Workflow and System Demonstration

To bridge the gap between fashion back-end compatibility modeling and real user experience, a basic but lively web-based User Interface (UI) was built. The UI is programmed with Dash in Python and includes functionalities for uploading items, prediction of outfit compatibility, weak point diagnosis, suggestion of improvement, and retrieval of external shopping recommendations through semantic search. The following examples illustrate the end-to-end process of the system, from outfit assembly to result interpretation and shopping recommendation.

A. Outfit Diagnosis and Bottom Substitution

Figure 12: User Interface - Diagnosis and Improved Bottom Recommendation

In this example, Figure 12, the user uploaded a complete five-piece outfit. After clicking "Check Compatibility," the model examined the original outfit, giving a relatively high initial score of 0.8431. The diagnosis, however, identified the Bottom (pants) as the item reducing compatibility. The new outfit was automatically created by substituting the bottom with a more visually compatible plaid skirt, improving the revised score to 0.9198. Furthermore, the proposed additional plaid skirt options obtained using SerpAPI allowed the user to filter or shop options. Thumbs-up/down ratings were collected for the suggestion overall as well as for each product suggestion to enable future personalization.

B. Footwear and Accessory Optimization

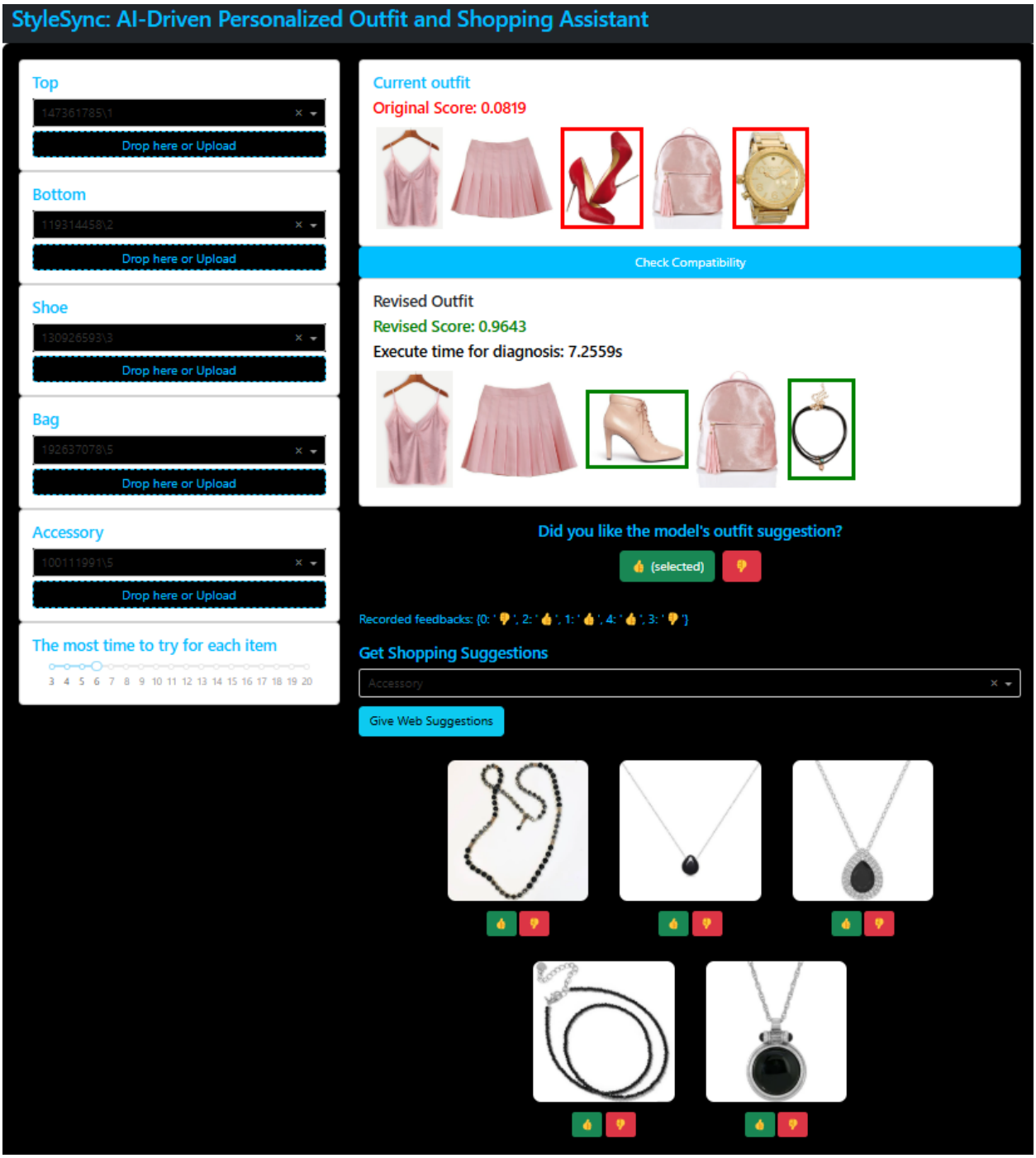

Figure 13: User Interface - Shoe and Accessory Correction for Stylistic Balance

The workflow in Figure 13 represents a more challenging situation where the initial compatibility score was low (0.0819), indicating a thematic mismatch between the uploaded shoes and the rest of the pink-colored outfit. The diagnosis module replaced the shoes with a beige ankle boot and also defined the accessory more clearly by suggesting a lighter, stylistically more suitable necklace. This resulted in a revised compatibility score of 0.9643. The user was able to examine and verify the outfit enhancement made by the system, while additional necklace suggestions were created on the fly to enable accessory styling refinement.

C. Complete Styling Overhaul

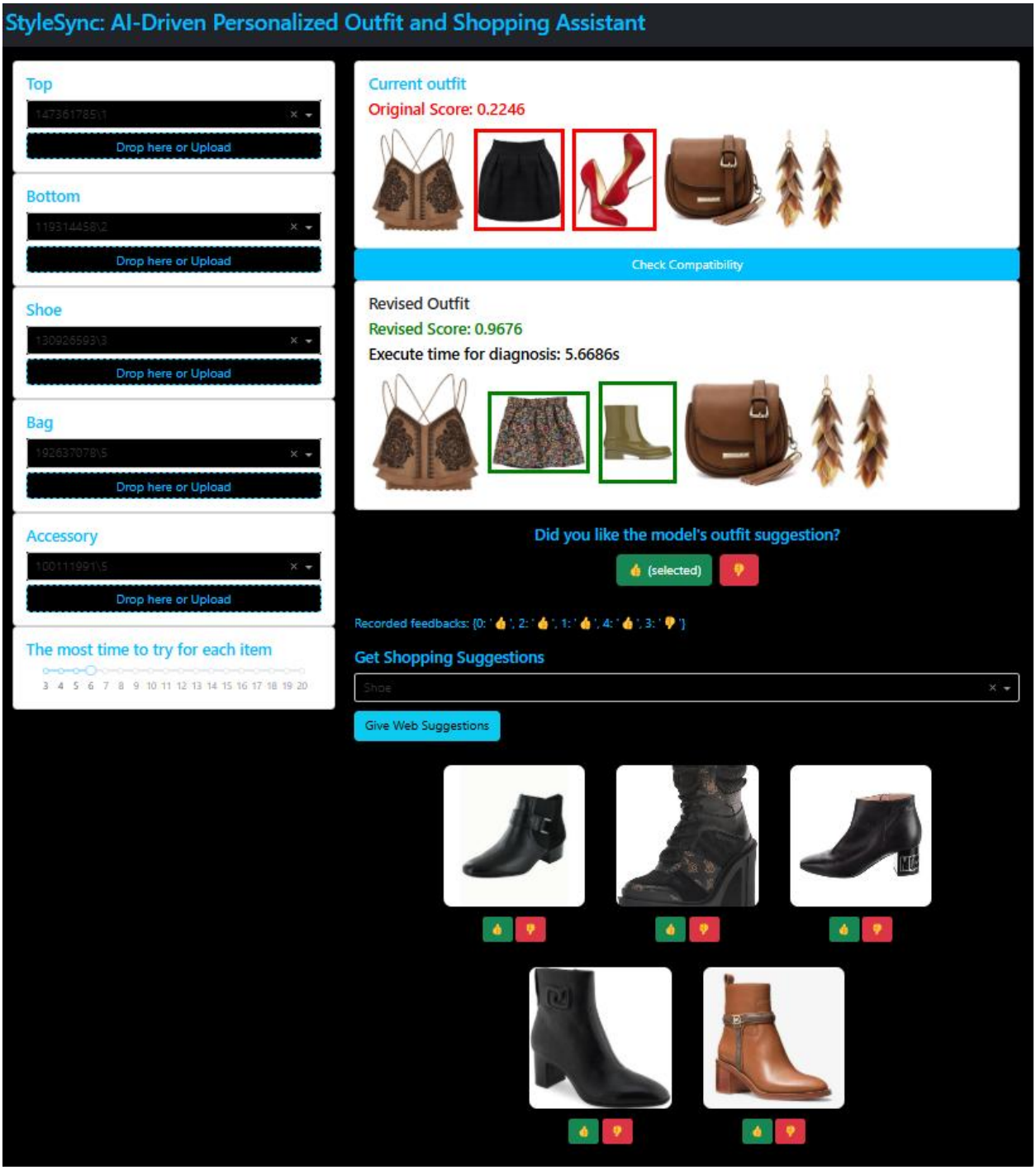

Figure 14: User Interface - Multi-Item Substitution for Full Style Coherence

Here, the initial outfit had shoe and bottom mismatch with the intended casual look in Figure 14, with an initial score of 0.2246. The model swapped the bottom with a printed skirt and replaced the heeled shoes with casual boots, radically improving the compatibility score to 0.9676. These multi-layered edits illustrate how the system can detect multiple incompatibilities simultaneously and recommend comprehensive, context-sensitive improvements between clothing items. The shopping suggestions offered boots that were consistent with the corrected casual look, allowing users to explore more purchasing ideas.

D. Naturally Compatible Outfit

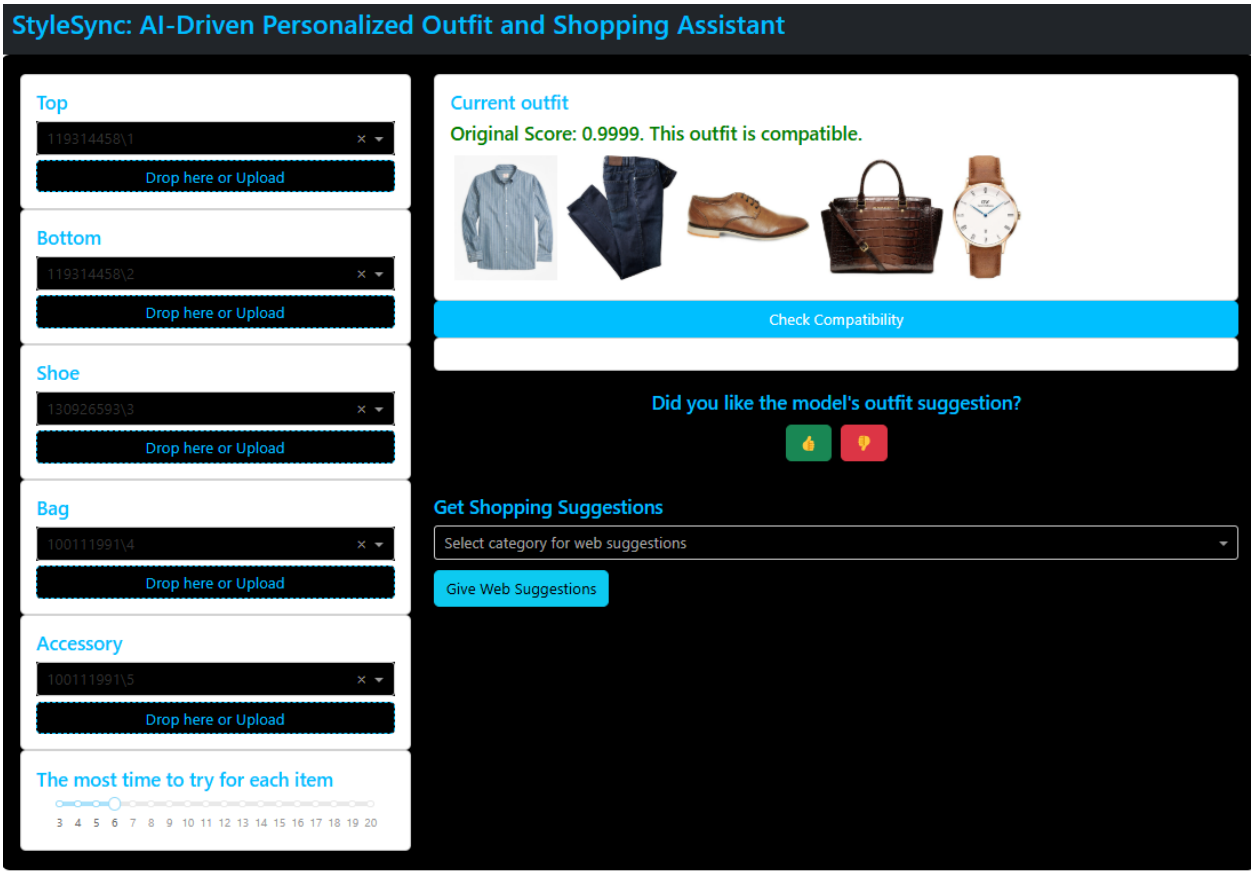

Figure 15: User Interface - High Compatibility Example without Revisions

This Figure 15 example presents a naturally well-coordinated outfit that the user uploaded. The model produced an excellent initial compatibility score of 0.9999, reflecting that no significant corrections were needed. Everything posted, shirt, jeans, shoes, bag, and accessory, made up a cohesive casual-to-business-casual ensemble, illustrating texture, structure, and stylistic balance. No exchanges were recommended, and web-based suggestions were not required, indicating the model's ability to not only diagnose problems but also recognize quality styling when it existed.

Section 5 – Conclusions and Future Work

5.1 Conclusions

The core aim of this project was to develop an efficient, intelligent system for fashion outfit compatibility assessment along with enhancing users' style by means of real-time and autonomous recommendations and proposals. Taking the ImageNet-pretrained ResNet-50 as the initial backbone and adding to it a Multi-Layered Comparison Network (MCN)-style extension, the model efficiently learned fine relational hints among objects within an outfit and at multiple abstraction levels varying from low-level textural resemblances to high-level thematic consistency.

Using the full supervision of learning paradigm, the model, when trained over the Maryland Polyvore Images Dataset, performed firmly both in the areas of compatibility scoring and fill-in-the-blank (FITB) activities. The module for diagnosis provided useful feedback through identification of weak spots in a set of attire, thereby filling the gap from plain prediction towards intelligent stylistic recommendations. With integration with external User Interface and API systems like SerpAPI, the platform also made it possible for users to obtain not only outfit reviews, but also personalized shopping suggestions based on the semantic understanding of fashion items.

The feedback loop enabled the system to record user preferences for future personalization, and the UI provided an end-to-end seamless experience from outfit upload to diagnosis and dynamic web recommendations. Overall, this project achieved a highly functional, scalable solution that moves toward making AI-driven personal styling both accessible and intuitive for real-world users.

5.2 Future Work

Although the current system is resilient, there are several possible improvements that can be pursued with more time, data, and computational resources:

- Large Language Model (LLM) Ingestion: Subsequent releases could include ingestion of models like GPT-4 or Fashion-oriented LLMs to generate free-text outfit suggestion generation. Rather than relying on formal semantic queries, the model could ingest outfit description, user mood, and occasion prompts and dynamically generate text-form and image-form recommendations. This would greatly improve styling flexibility and customization.
- Fine-tuning GPT descriptions: Fine-tuning or training a GPT model on highly curated fashion datasets might make it possible to generate rich style recommendations, i.e., "Wearing the structured leather jacket with denim maintains an urban casual look." This would enrich system responses and make them more end-user-understandable.
- Smart API Integration with Context-Based Filtering: More advanced integration with APIs, rather than mere retrieval of products, may have the ability to make multi-parameter searching feasible — by style (e.g., "sneakers on a minimalist for under $100") and not just by color or type and user-preferred behavior gathered over time. Occasion- (or season-) based or trend-based retrieval informed by context awareness would increase recommendations significantly in terms of quality.
- Learning from User Feedback: The thumbs-up/down feedback model already captures preference signals, but maybe future work would be to retrain models or fine-tune retrieval weights on this feedback. Active learning loops would likely make the system progressively smarter and more responsive to individual preferences.
- Scaling Multimodal Reasoning: Future work could incorporate textual metadata (product names, reviews, tags) deeper into visual features, building a stronger multimodal matching system that understands what something is and what it looks like .
- Processing Out-of-Distribution Items: Another promising direction would be to train few-shot or zero-shot compatibility models that can process new fashion types or novel items not encountered during training.

References

[1] S. Shirkhani, H. Mokayed, R. Saini, and H. Y. Chai, "Study of AI-Driven Fashion Recommender Systems," SN Computer Science, vol. 4, no. 5, Jul. 2023, doi: 10.1007/s42979-023-01932-9.

[2] Y. Zhang and J. Caverlee, "Instagrammers, Fashionistas, and Me," Nov. 2019, doi: https://doi.org/10.1145/3357384.3358042.

[3] B. Suvarna and S. Balakrishna, "Enhanced content-based fashion recommendation system through deep ensemble classifier with transfer learning," Fashion and Textiles, vol. 11, no. 1, Jul. 2024, doi: 10.1186/s40691-024-00382-y.

[4] Y. Lin, P. Ren, Z. Chen, Z. Ren, J. Ma, and M. De Rijke, "Explainable Outfit Recommendation with Joint Outfit Matching and Comment Generation," IEEE Transactions on Knowledge and Data Engineering, vol. 32, no. 8, pp. 1502–1516, Mar. 2019, doi: 10.1109/tkde.2019.2906190.

[5] Y.-L. Lin, S. Tran, and L. S. Davis, "Fashion outfit Complementary item retrieval," 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Jun. 2020, doi: 10.1109/cvpr42600.2020.00337.

[6] A. Kalinin, A. A. Jafari, E. Avots, C. Ozcinar, and G. Anbarjafari, "Generative AI-based style recommendation using fashion item detection and classification," Signal Image and Video Processing, Sep. 2024, doi: 10.1007/s11760-024-03538-x.

[7] X. Wang, B. Wu, and Y. Zhong, "Outfit Compatibility Prediction and Diagnosis with Multi-Layered Comparison Network," Oct. 2019, doi: https://doi.org/10.1145/3343031.3350909.

[8] K. Cui et al., "Correlation-Aware cross-modal attention network for fashion compatibility modeling in UGC systems," ACM Transactions on Multimedia Computing Communications and Applications, Oct. 2024, doi: 10.1145/3698772.

[9] X. Han, Z. Wu, Y.-G. Jiang, and L. S. Davis, "Learning Fashion Compatibility with Bidirectional LSTMs," Proceedings of the 30th ACM International Conference on Multimedia, Oct. 2017, doi: 10.1145/3123266.3123394.

[10] "Polyvore outfit dataset," Kaggle, Mar. 07, 2025. https://www.kaggle.com/datasets/enisteper1/polyvore-outfit-dataset

[11] Xthan, "GitHub - xthan/polyvore-dataset: Dataset used in paper 'Learning Fashion Compatibility with Bidirectional LSTMs,'" GitHub. https://github.com/xthan/polyvore-dataset/

[12] "Maryland Polyvore Images," Kaggle, Jul. 14, 2020. https://www.kaggle.com/datasets/dnepozitek/maryland-polyvore-images/data

[13] K. He, X. Zhang, S. Ren and J. Sun, "Deep Residual Learning for Image Recognition," 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 2016, pp. 770-778, doi: 10.1109/CVPR.2016.90.